

Лабораторная работа №5

“Решение СЛАУ”

Цель работы: изучение методов решения СЛАУ.

Ход работы:

№	Описание проделанных действий	Результат этапа
1	Ознакомление с теоретическими сведениями о прямых и итерационных методах решения СЛАУ. Анализ метода LU-разложения, методов Якоби и Гаусса-Зейделя. Изучение требований лабораторной работы.	Сформирован четкий алгоритм реализации; выбраны программные инструменты (Python, библиотеки NumPy, Pandas, Matplotlib, timeit) для решения системы, анализа времени выполнения и визуализации результатов.
2	Реализация функции решения СЛАУ методом LU-разложения. Определение времени выполнения алгоритма. Сравнение аналитического решения системы и решения, полученного с помощью созданной функции.	Созданная функция успешно решила систему уравнений. Аналитическое решение $[1, -1, 0, 1]$ полностью совпало с численным.
3	Реализация функций итерационных методов Якоби и Гаусса-Зейделя с покомпонентным расчётом. Проведение вычислительных экспериментов для значений точности от 10^{-1} до 10^{-9} .	Оба метода успешно сошлись к решению $[1, -1, 0, 1]$.
4	Сравнение количества итераций методов Якоби и Гаусса-Зейделя для различных значений точности. Построение графика зависимости количества итераций от точности.	Метод Гаусса-Зейделя показал более высокую скорость сходимости: для точности 10^{-9} потребовалось около 15 итераций против 25 итераций у метода Якоби.
5	Сравнение времени выполнения методов Якоби и Гаусса-Зейделя	Время выполнения обоих методов возрастает с повышением

	для различных значений точности. Построение графика зависимости времени выполнения от точности.	требований к точности. Метод Гаусса-Зейделя выполняется быстрее за счёт меньшего количества итераций при одинаковой точности.
6	Реализация матричной версии метода Якоби с использованием операций библиотеки NumPy. Сравнение времени работы матричной и поэлементной реализаций.	Матричная реализация оказалась примерно в 2.13 раза быстрее поэлементной для матрицы размером 4×4. Преимущество обусловлено использованием оптимизированных линейно-алгебраических операций NumPy.
7	Анализ полученных результатов и формулирование выводов. Сравнение прямого и итерационных методов. Оценка явления сходимости итерационных методов в зависимости от свойств матрицы.	Сформулированы выводы по результатам работы. Проведён сравнительный анализ прямого и итерационных методов, оценено влияние свойств матрицы на сходимость, а также эффективность различных реализаций методов.

Выводы:

- Прямой метод LU-разложения даёт точное решение за фиксированное число операций и не зависит от свойств матрицы, однако при больших размерностях может быть вычислительно затратным. Итерационные методы (Якоби и Гаусса-Зейделя) требуют меньше памяти, но их сходимость напрямую зависит от свойств матрицы.
- Метод Гаусса-Зейделя показал более высокую скорость сходимости: для достижения точности 10^{-9} ему потребовалось около 15 итераций, тогда как методу Якоби - около 25 итераций. Это объясняется тем, что в методе Гаусса-Зейделя каждая компонента вектора обновляется с использованием уже вычисленных значений на текущей итерации, что ускоряет сходимость. Соответственно, время выполнения метода Гаусса-Зейделя также оказалось меньше.
- Матричная реализация метода Якоби, использующая операции библиотеки NumPy, оказалась примерно в 2.13 раза быстрее поэлементной реализации с вложенными циклами. Для матрицы размером 4×4 ускорение не является критическим, однако для систем большей размерности преимущество

матричного подхода становится существенным за счёт использования оптимизированных линейных алгебраических операций.

Использованные в работе функции:

np.array()	Создание массива из списка	object - входные данные; dtype - тип данных
np.zeros()	Создание массива из нулей	shape - размерность
np.ones()	Создание массива из единиц	shape - размерность
np.full()	Создание массива с заданным значением	shape - размерность; fill_value - значение
np.identity()	Создание единичной матрицы	n - размер матрицы
np.diag()	Извлечение диагонали матрицы	v - матрица
np.diagflat()	Создание диагональной матрицы	v - входные данные
np.sum()	Сумма элементов массива	a - массив
np.sqrt()	Квадратный корень	x - массив или число
np.isclose()	Сравнение массивов с допуском	a, b - массивы
np.random.seed()	Фиксация генератора случайных чисел	seed - начальное число
np.linalg.norm()	Норма вектора	x - входные данные
timeit.timeit()	Замер времени выполнения	stmt - код; globals - переменные; number - число запусков
plt.figure()	Создание окна графика	figsize - размер
plt.plot()	Построение линейного графика	x, y - данные; marker - маркер; label - подпись
plt.semilogy()	График с лог-шкалой по Y	x, y - данные; marker - маркер; label - подпись
plt.xlabel()	Подпись оси X	xlabel - текст
plt.ylabel()	Подпись оси Y	ylabel - текст

plt.title()	Заголовок графика	label - текст
plt.legend()	Отображение легенды	loc - расположение
plt.grid()	Добавление сетки	True/False
plt.tight_layout()	Настройка отступов	---
plt.show()	Отображение графиков	---
pd.DataFrame()	Создание таблицы	data - данные; columns - названия столбцов
df.to_string()	Преобразование в строку	index - включать индексы
np.copy()	Создание копии массива	a - массив